One-Page PRD: Wise-Edge-LR (저사양·Edge 기반 LLM/RAG 시스템)

비용절감을 위한 저사양 또는 Edge 기반의 LLM/RAG 시스템 구축 Establishment of low-spec or Edge-based LLM/RAG systems for cost reduction

1. 개요

- **한 줄 요약**: 상용 LLM API(ChatGPT·Gemini 등) 호출 비용·토큰 과금·데이터 외부유출 부담을 없애기 위해, Apple Silicon(MLX)·저사양 서버(Ollama/llama.cpp/vLLM)에서 구동되는 **온프레미스 Standalone LLM + RAG** 시스템을 구축한다. 1차로 사내 한글 문서 Q&A를 제공하고, 도메인·기능 확장이 가능한 구조로 설계한다.
- **배경 / 왜 지금**: 클라우드 LLM API는 백만 토큰당 입력 $2.5~15 / 출력 $10~75 수준(GPT-4o $5/$20, Claude 3 Opus $15/$75) 으로 사용량 증가 시 비용·예측가능성이 악화되고, 민감 사내문서를 외부 API에 보내야 하는 거버넌스 리스크가 있다(근거: `.docs/Perplexity-LLM API 서비스 비용 및 성능 비교 추천.md`). 오픈소스 모델 온프레미스 배포 시 토큰 비용은 사실상 0 또는 GPU 기준 ~$0.35/백만토큰 수준으로 내려간다.
- **작성자 / 날짜 / 버전**: bclim@edentns.com / 2026-06-14 / v0.1 (draft)

2. 문제와 대상 (Why & Who)

- **해결하려는 핵심 문제**: 상용 LLM API의 (1) 토큰 종량 과금으로 인한 누적 비용, (2) 사내·고객 문서의 외부 전송 데이터유출 리스크, (3) 외부 서비스 의존(가용성·정책변경) 을 제거한다.
- **비즈니스 관점 중요성**: API 운영비를 자본성 하드웨어 1회 투자 + 저전력 운영비로 전환해 사용량과 무관하게 비용을 고정·예측 가능하게 만든다. 데이터를 망 내부에 둬 규제(PIPA/ISMS-P/CSAP)에 정합한다.
- **근거 데이터 / VOC / 로그**: 가정: 현행 상용 API 월 사용량·비용 실측치 미확보 — 도입 전 1개월 토큰/비용 baseline 수집 필요(Open Question).
- **타깃 사용자 / 페르소나**:
  - 1차: **사내 직원** — 한글 사내 문서/규정/매뉴얼을 자연어로 검색·질의(온프레미스, 외부유출 0).
  - 2차: 공공/govtech 담당자 → (확대) 금융·헬스케어 → (확대) 제품(식품) 등 도메인.
- **대표 사용 상황**: 직원이 "○○ 규정 최신 기준 알려줘"를 입력하면, 사내 문서 코퍼스에서 근거 구절을 검색해 출처와 함께 한글로 답한다. 외부 망 호출 없음.

3. 범위 (What — In / Out)

이번 릴리즈(v1) 핵심 기능 (P0/P1)

#	기능	우선순위
1	**문서 인제스천 + 청킹 + 임베딩** — 업로드 → 청킹 → 벡터DB(pgvector) 인덱싱 (한글 문서/PDF/CSV)	P0
2	**RAG 검색 + LLM 응답(스트리밍)** — 검색증강 질의응답, 출처 표기, 토큰 스트리밍	P0
3	**웹 UI** — React 채팅 UI 또는 Open WebUI 포크	P0
4	**비용/토큰 모니터링** — 로컬 vs 상용 API 환산 비용 비교, 토큰 카운팅 대시보드	P1
5	**모델 런타임 추상화** — MLX(edge)·Ollama·llama.cpp·vLLM standalone 백엔드 교체	P1

하지 않을 것 (Out of Scope, v1)

- Foundation model **사전훈련**(from-scratch) — 사전학습 모델 사용/파인튜닝만(근거: `.docs/Perplexity-Foundation model 구축...md` 는 참고 자료, v1 비범위).

- 멀티모달(이미지/음성) 입력, 자동 파인튜닝·지속학습 파이프라인.
- 2차 도메인(금융/헬스케어) 전용 규제 기능 — 별도 릴리즈.
- 고가 GPU 클러스터 전제 구성(저사양·edge 가 전제).

4. 사용자 경험 (How — User Flow)

- **시나리오 1 (질의응답)**: 직원이 웹 UI에 한글 질문 입력 → 백엔드가 질문 임베딩 → pgvector 유사도 검색 top-k → 검색 구절 + 질문을 로컬 LLM(Ollama/MLX) 프롬프트로 구성 → 토큰 스트리밍으로 답변 + 출처 표시.
- **시나리오 2 (문서 등록)**: 관리자가 문서(PDF/한글/CSV) 업로드 → 청킹(문단/토큰 단위) → 한글 임베딩 모델로 벡터화 → pgvector 저장 → 즉시 검색 대상에 포함.

5. 성공 지표 (Success)

KPI	Baseline (가정)	Target	측정 방법
LLM 운영 비용 절감률	상용 API 월비용 (도입 전 1개월 실측, 가정:)	**≥ 70% 절감**	로컬 토큰량 × 상용 단가 환산 vs 실청구액(모니터링 대시보드)
응답 첫 토큰 지연(TTFT)	가정: 측정 전	≤ 2s (저사양 타깃)	서버 로그 타임스탬프
생성 속도	가정: 모델·HW별	sLLM 4bit 양자화 기준 실용 tok/s 확보	벤치 스크립트(근거: `.docs/Perplexity-Local LLM으로 ollama...응답속도.md`)
검색 적합성	—	답변 출처 정확률 사용자 평가 ≥ 목표	샘플 질의 셋 수동 평가
데이터 외부 전송	—	**0건** (온프레미스 강제)	네트워크 egress 점검

> 업종 오버레이 제안(`domains/ict-saas.yaml`, `domains/govtech.yaml`)된 **검증 가능한 수용 기준 후보**: 크로스테넌트 데이터 누출 차단(RLS), OpenAPI 계약 적합성, 감사로그 보관·삭제 증명(ict-saas); 고유식별정보 암호화·마스킹 검증, KWCAG 2.2 접근성 점검, 공공데이터 API 적합성(govtech).

6. 리스크 및 메모

- **주요 리스크**:
  - 저사양/edge 에서 대형 모델 구동 시 응답속도·품질 저하 → 양자화(4bit GGUF/MLX)·작은 모델·스트리밍으로 완화.
  - 한글 토큰화/임베딩 품질 → 한글 특화 토크나이저·임베딩 모델 선정 필요(근거: `.docs/Perplexity-한글 문서를 기반으로 RAG...md`, `.docs/Perplexity-LLM...토큰을 계산하는 방법...md`).
  - 다중 런타임(MLX/Ollama/llama.cpp/vLLM) 동시 지원 → 추상화 계층 복잡도 증가.
- **의존성**: pgvector(PostgreSQL), Ollama/MLX 런타임, 임베딩 모델, (선택) Open WebUI 포크, Docker.
- **미결정 사항 (Open Questions)**:
  1. 도입 전 상용 API 비용 baseline 실측치?
  2. 1차 타깃 하드웨어 사양 확정(Mac mini/Studio MLX vs x86 서버)?
  3. 기본 LLM·임베딩 모델 후보(한글 성능 vs 메모리) 선정?
  4. UI: React 자체구현 vs Open WebUI 포크?

---

7. 기술 메모 (Technical Notes) — 예시/초안

- **아키텍처 (초안)**: 웹 UI(React/Open WebUI) → 백엔드(Express.js 또는 FastAPI, RAG 오케스트레이션 + LLM 게이트웨이/미들웨어) → LLM 런타임(Ollama / MLX / llama.cpp / vLLM standalone) + 벡터DB(PostgreSQL + pgvector). LLM 미들웨어로 백엔드를 백엔드 런타임에서 분리해 모델 교체·라우팅을 흡수(근거: `.docs/Perplexity-Local Ollama 미들웨어.md`, `.docs/Perplexity-Langchain.js...RAG...md`).

- **RAG 파이프라인**: 문서 로더 → 청킹(문단/고정토큰/오버랩) → 한글 임베딩 → pgvector 저장 → 유사도 검색 top-k → 프롬프트 합성 → LLM 스트리밍 응답(근거: `.docs/Perplexity-RAG 시스템 구축, 문서를 청킹...md`, `.docs/Perplexity-주요 RAG 프레임워크 및 장단점 비교.md` — LangChain/Haystack/RAGFlow 비교).
- **프레임워크 후보**: RAG 오케스트레이션 = LangChain(.js) 우선(빠른 프로토타입·온프렘 보안제어), 프로덕션 확장 시 Haystack 검토.
- **비기능 요구사항(업종 오버레이 반영)**:
  - ict-saas: 멀티테넌시(RLS→스키마/DB-per-tenant), 관측성(OpenTelemetry), API-first(OpenAPI 계약), 감사로그 보관·삭제(ISMS-P/PIPA).
  - govtech: 망분리→N2SF 등급(C/S/O), CSAP 인증 공공클라우드(2026 완화), KWCAG 2.2 웹접근성, 주민 PII 암호화·마스킹, eGovFrame 정합(필요 시).
  - SLO/성능: 저전력 운영·고정비, TTFT/생성속도 목표(§5).
- **데이터 / 로깅 요구사항**: 데이터 등급 — 테넌트/사내 문서(민감), PII(규제대상), 감사로그/텔레메트리(내부, 보관·삭제정책). 토큰/비용·질의 로그를 §5 KPI 측정용으로 수집.
- **외부 연동 / API**: 기본 외부 호출 0(온프레미스). 내부 REST/스트리밍(SSE/WebSocket) API. 선택적으로 상용 API 폴백은 비범위.
- **리스크 / 트레이드오프**: 저사양 비용절감 ↔ 모델 품질/속도. 양자화·작은모델·캐싱으로 절충(근거: `.docs/Perplexity-MLX Apple sLLM Tutorial.md`, `.docs/Perplexity-Local LLM(Ollama) 설치...md`).

8. 실행 연결 (Epic / Story 힌트)

- **Epic A — 런타임 기반**: Ollama/MLX/llama.cpp/vLLM standalone 구동 + LLM 미들웨어 추상화 + Docker compose.
- **Epic B — RAG 인제스천**: 문서 로더·청킹·한글 임베딩·pgvector 인덱싱(PDF/한글/CSV).
- **Epic C — 질의응답**: RAG 검색 + 프롬프트 합성 + 스트리밍 응답 + 출처 표기.
- **Epic D — 웹 UI**: React 채팅 또는 Open WebUI 포크, 문서 업로드 화면.
- **Epic E — 비용/토큰 모니터링**: 토큰 카운팅, 로컬 vs 상용 환산 비용 대시보드.
- **Epic F — 도메인/규제 확장**: 1차 ict-saas/govtech 정합 → 2차 금융·헬스케어 → 제품(식품) 확대.

참고 (근거 출처)

- `.docs/` Perplexity 리서치 14종(LLM 비용/토큰, Ollama·MLX·llama.cpp 로컬 LLM, RAG 프레임워크·청킹·Vector DB, 한글 RAG, Open WebUI, LangChain.js RAG, 미들웨어).
- `domains/ict-saas.yaml`, `domains/govtech.yaml` (KSIC J/O 규제·데이터등급·검증기준 후보).